

AI Agents: Who's Actually Doing the Work?



香港城市大學

City University of Hong Kong

Innovating into the Future

IS5542 — Lecture 3 — AI Agents

SIR DEMIS HASSABIS: THE ARCHITECT OF AGI

Co-founder & CEO, Google DeepMind | **Nobel Laureate** | Mission: Solve intelligence, and then use that to solve everything else.

QUICK FACTS

Born: July 27, 1976 (London, UK)

Heritage: Greek Cypriot (Father) & Chinese Singaporean (Mother)

Education: Double First, Cambridge (Computer Science);
PhD, UCL (Cognitive Neuroscience)

IQ/Skillset: Chess Master at age 13; 5-time World Games
Champion (Mind Sports Olympiad)



DEEPMIND'S GREATEST HITS

AlphaGo (2016):

First AI to defeat a Go world champion (Lee Sedol) – a feat thought decades away.



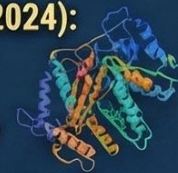
AlphaZero (2017):

Mastered Chess, Shogi, and Go from scratch in hours, self-taught strategies.



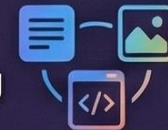
AlphaFold (2020-2024):

Solved the "protein folding problem", predicting structure of nearly all known proteins.

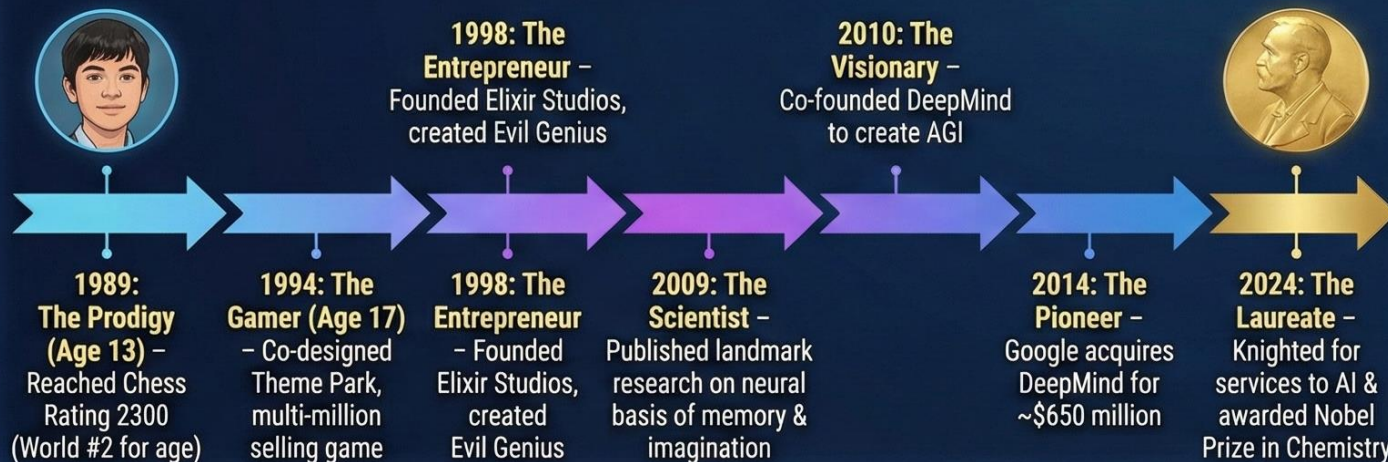


Gemini (2023-Present):

Leading Google's charge into multimodal, large-scale AI models, competing at GenAI frontier.



THE EVOLUTION OF A GENIUS (Timeline)



KEY AWARDS & RECOGNITION

"He is likely the most decorated AI researcher in history."

- **Nobel Prize in Chemistry (2024)** – For protein structure prediction.
- **Lasker Award (2023)** – Often called 'America's Nobel'.
- **Breakthrough Prize (2023)** – In Life Sciences.
- **Knight Bachelor (2024)** – Bestowed by the British Crown.
- **Fellow of the Royal Society (FRS)** – Joined the ranks of Newton and Darwin.

NOTABLE QUOTE

"AI is the ultimate tool for accelerating scientific discovery. It's the Apollo Program of our time."

Demis's view on AGI

Proprietary + Confidential



Google

https://youtu.be/BblaYFHxW3Y?si=p4ow0s2mdnHFcz_n&t=419

World Models

Proprietary + Confidential



Google

<https://youtu.be/PqVbypvxDto?si=4ezY3qrqWOHie3aR&t=1064>

Lecture roadmap

1 Foundations and agency

What makes a system an agent?

2 The agent system

How do the loop and harness support action?

3 Agent architectures

How do Deep Research, OpenClaw and autoresearch organize work?

4 Applications and scientific AI

Where do agents work, and what do scientific AI cases show?

5 Evaluation and deployment

What evidence supports a launch decision?

6 Synthesis and discussion

What have we learned, and how would we design a bounded task?

The model didn't change. The score did.

6.7%

SWE-bench, before



68.3%

SWE-bench, after — same model

Grok Code Fast's score moved 10x when only the edit-tool format changed. No retraining. Vercel found removing 80% of their agent's tools beat upgrading the model.

So when a vendor says “our agent,” how much of that claim is the model, and how much is the scaffolding built around it? That is the question this lecture is organised around.

And the Gap between model capability and real-world Trust Is Widening

24.2 pp

lower human PR-merge rate than benchmark-passing implies (METR)

~25%

pass rate on τ -knowledge — messy internal enterprise support tasks

20%

of organisations report scaling agents enterprise-wide (McKinsey, 2026)

+9.6 pp/yr

how fast that gap is widening — not shrinking

~40%

pass rate even when handed the correct source documents

6%

qualify as “high performers” from that investment

Four numbers, one shape: capability is climbing benchmarks faster than it is earning trust, documents, or enterprise scale. Keep these on a mental shelf — we return to each one.

1. Foundations and agency

1. Foundations and agency

A History Longer Than the Technology

The agent idea starts before the word — and most of its vocabulary was fixed decades before an LLM could act on it.

Foundations: 1943–1970

Formal agent theory: 1970s–1990s

Learning agents & deep RL: 2000s–2010s

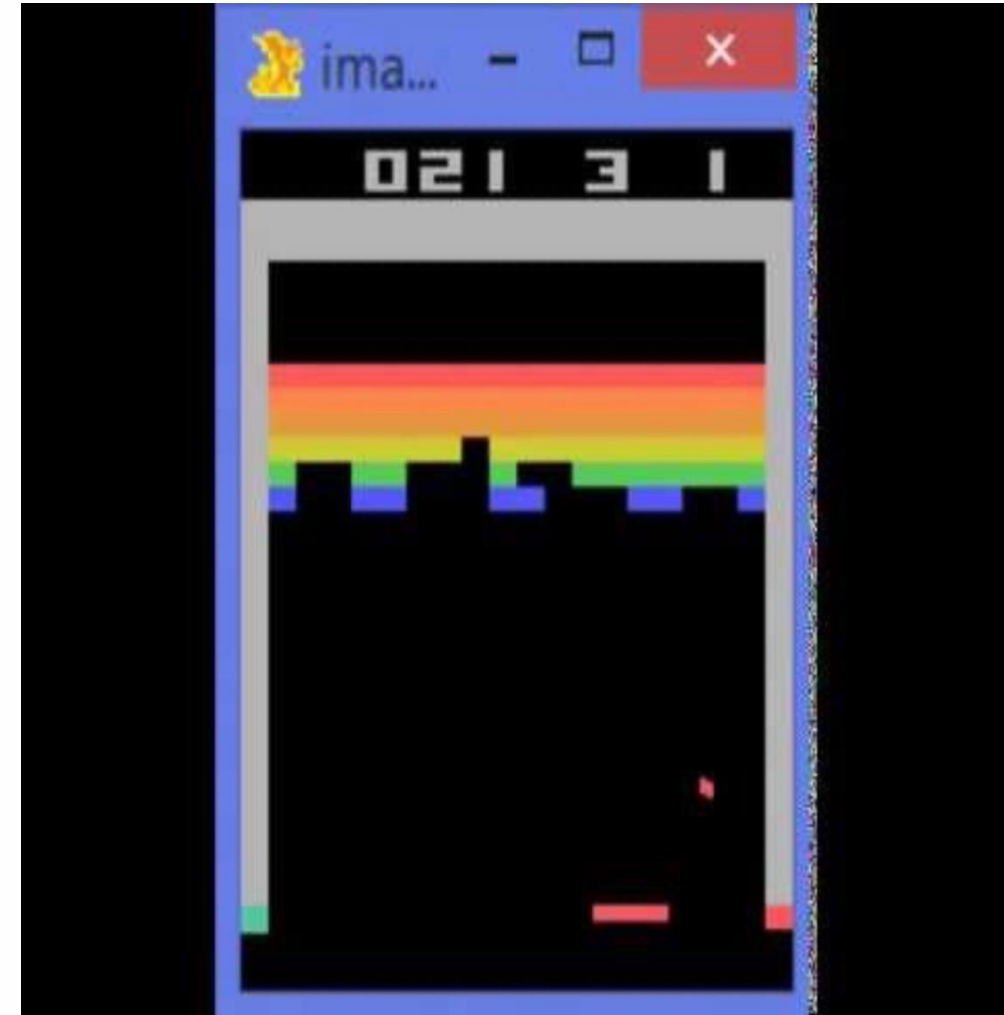
The LLM turn: 2022–2023

Productisation & protocols: 2024–2025

Maturation: 2026

Learning Agents & Deep RL: 2000s–2010s

- Cognitive architectures (ACT-R, Soar) integrate memory, reasoning, and action into multi-step cognition.
- The decisive shift: reinforcement learning meets deep neural networks. DeepMind's Deep Q-Network (2013) is the first artificial agent to combine representation learning with RL end to end — learning Atari from pixels.
- AlphaGo and its successors extend this to long-horizon planning under uncertainty.
- These systems were genuinely agentic — they perceived, planned, and acted — but each was trained for a single environment. They could not be told what to do in words, and could not transfer.



Productisation & Protocols: 2024–2025



- Mar 2024 — Cognition's Devin packages an autonomous software engineer with a shell, editor, and browser in a sandbox.
- Oct 2024 — Anthropic's computer use beta lets a model see a screen, move a cursor, click, and type — generalising the tool interface to any GUI software.
- Nov 2024 — Anthropic open-sources the Model Context Protocol (MCP). Adoption is unusually fast: Microsoft (Mar 2025), OpenAI (Mar 2025), Google (Apr 2025); donated to the Agentic AI Foundation Dec 2025.
- Apr 2025 — Google announces Agent2Agent (A2A); donated to the Linux Foundation that June. A2A reaches v1.0 in March 2026 with 150+ organisations reported as adopters.

\$8.7B

raised by agent startups in Q1 2025 alone — up 143% year over year

Consumer & enterprise products, same window: OpenAI's Operator and Deep Research (Jan 2025), Manus, Agents SDK & Responses API (Mar 2025), AWS Bedrock AgentCore (GA Oct 2025).

Maturation: 2026

- Coding agents become the proving ground; public SWE-bench Verified leaderboards report scores above 95% — though maintainers and independent evaluators warn the benchmark is mature and heavily represented in training data.
- METR's time-horizon work gives a more robust picture: the length of software task a frontier model completes with 50% reliability has been doubling roughly every 4–7 months, reaching ~5–6 hours of expert-equivalent work for leading models in early 2026.
- “Context engineering” replaces “prompt engineering”; harness engineering emerges as a named practice — the scaffolding around the model, not the weights, is now the dominant lever on reliability.

What Actually Makes Something an Agent?

Applying 30-year-old definitions to a brand-new substrate — and drawing the line between an agent and a workflow that just looks like one.

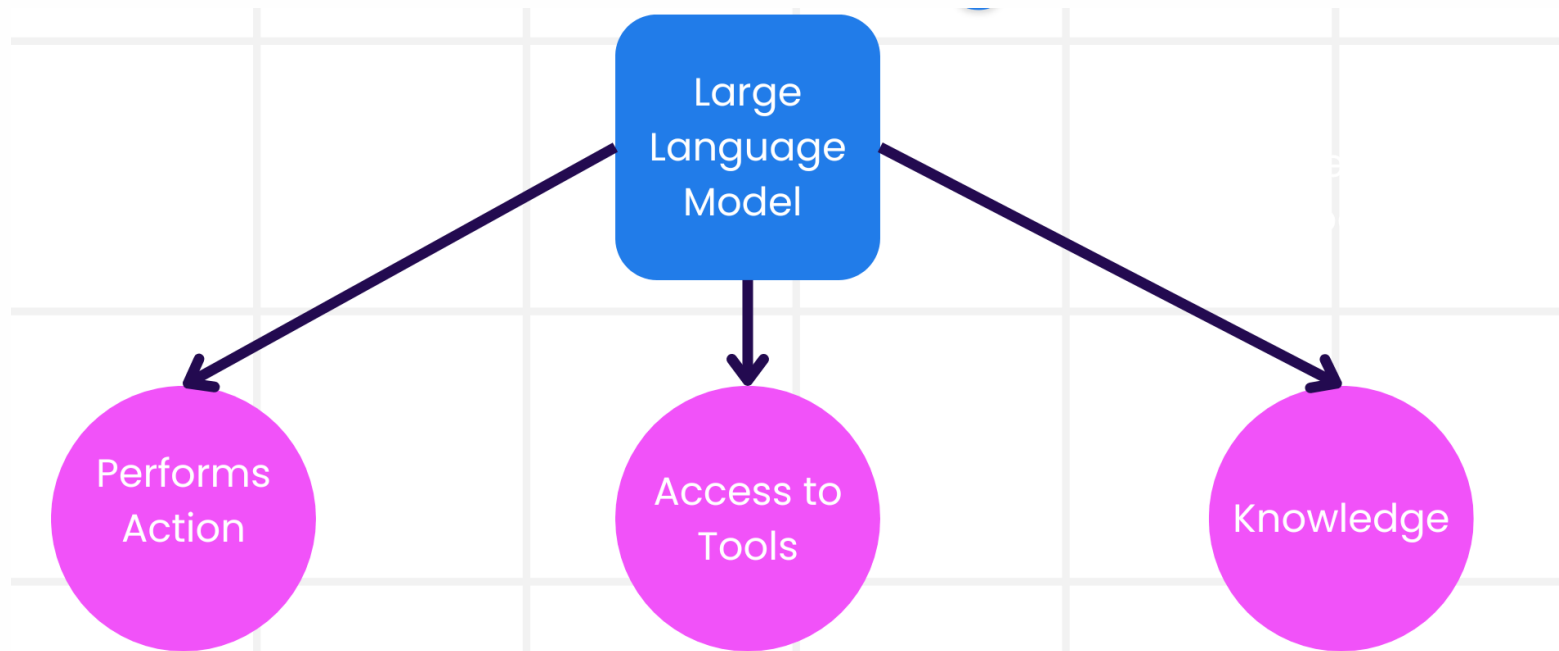
Four Pillars of Agency

Autonomy Levels

Hands of Agents (MCP)

Memory

What an AI agent does



Goal

Pursue a defined outcome.

Action

Choose tools and observe real results.

Control

Continue, stop or request help within limits.

The model helps choose actions. The application executes tools and enforces the limits.

Chatbots, workflows and agents

Approach	Who chooses the next step?	Travel example
Chatbot	User drives each turn	Explain baggage rules
Fixed workflow	Product logic defines the route	Search, filter and rank
Single agent	Model chooses within limits	Revise a search after failure
Multiple agents	A coordinator delegates tasks	Research independent destinations

A product may combine fixed workflow steps with agent decisions.

The Paradigm Shift: From Synthesis to Sovereignty

Generative AI (2023–2024)



- Probabilistic Content
- Request-Response Loop
- Passive Oracle
- Description: The era of describing the world. Systems operated on linear prompt-response cycles to synthesize existing information.

Agentic AI (The Present - 2026)



- Goal-Directed Autonomy
- Perception-Action Loop
- Active Operator
- Description: The era of changing the world. Systems utilize reasoning engines to orchestrate tools and alter the state of their environment.

The Four Pillars of Agency

Agentic AI is defined by the capacity to pursue complex, multi-step goals with limited intervention.



Goal-Directed Autonomy

Agents accept high-level objectives (e.g., 'Plan a campaign') rather than step-by-step instructions. They independently decompose objectives and prioritize actions.



Perception-Action Loops

A closed feedback loop:
Perceive → Reason → Act → Learn.
Agents perceive the environment, act to change it, and observe the result.



Tool Orchestration

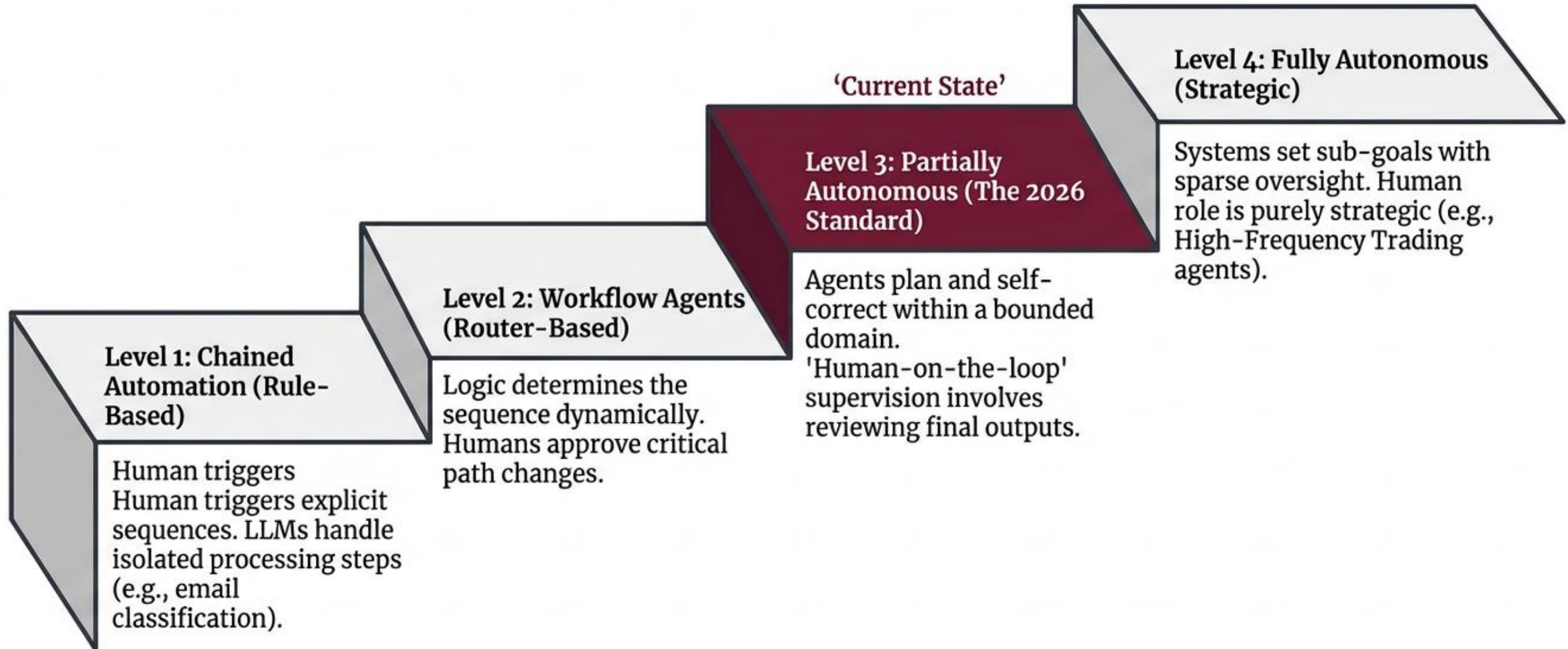
Decoupling intelligence from utility. Agents actively utilizing external tools (browsers, APIs) via protocols like MCP to execute tasks.



State & Persistence

The ability to maintain 'episodic memory.' Agents recall past actions and resume tasks over long horizons, overcoming the stateless nature of raw LLMs.

The Evolution of Autonomy Levels



2. The agent system

2. The agent system

The Harness

The runtime layer around the model — and the survey's central claim: it is now the primary determinant of whether an agent works.

What is a harness?

Why it matters more than it sounds

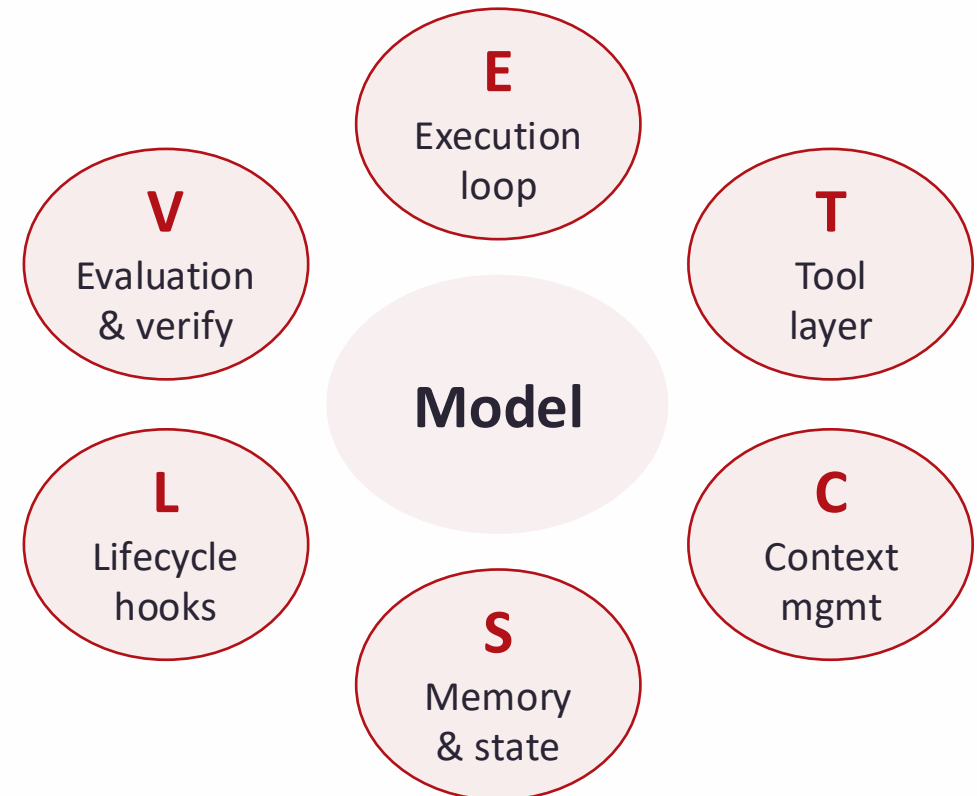
Nine components, component by component

One recurring lesson

What Is a Harness?

- A harness is the runtime layer around the model: the loop that keeps it running, the tools it can reach, the context it sees, the memory that outlives the session, the permissions that constrain it, the instrumentation that makes it debuggable.
- The model supplies judgement; the harness supplies everything else.
- A 2026 survey formalises the harness as a six-part tuple, $H = (E, T, C, S, L, V)$, and argues the harness — not the model — is the primary determinant of agent reliability at scale.

$$H = (E, T, C, S, L, V)$$



Why It Matters More Than It Sounds

6.7% → 68.3%

Grok Code Fast's SWE-bench score — harness edit-tool format changed, model unchanged. Tested across fifteen models.

–80% tools

Vercel found removing 80% of their agent's tools improved performance more than upgrading the model.

+90%, 15×

Anthropic's multi-agent research system beat single-agent by >90% — through orchestration, at ~15× token cost.

“If your agent is failing, the model is usually not the first thing to change.”

— the uncomfortable corollary

Case brief: a Tokyo trip



User and goal

Two travellers leave Hong Kong for a three-day Tokyo trip.

Constraints

HK\$8,000 for two return flights and one room for two nights. Dates are flexible.

MVP output

Two feasible options with sources and price timestamps. Meals and local transport excluded.

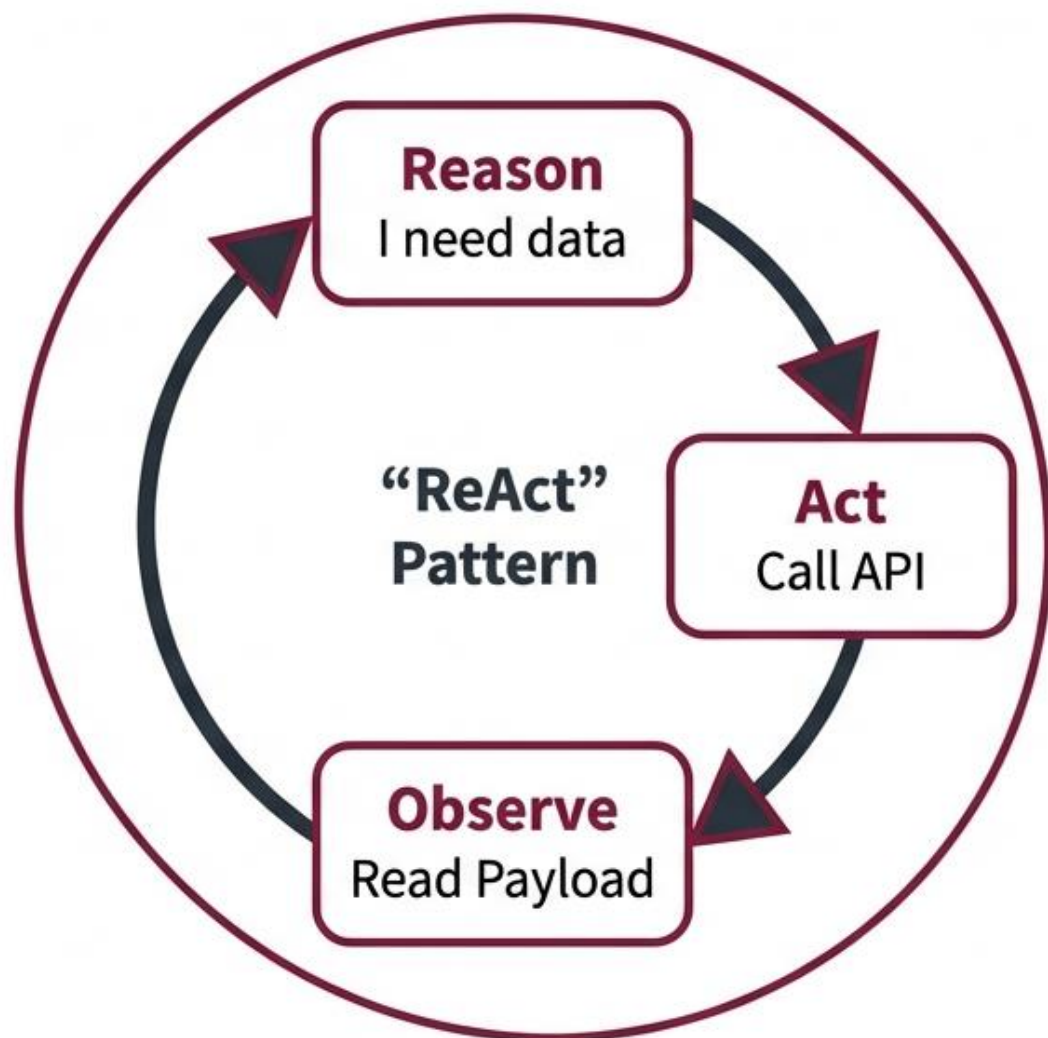
Completion boundary

The user accepts a plan. Calendar writes, booking and payment are future scope.

Fictional classroom brief. Budget feasibility must be checked with live data.

Flow Engineering: Designing the Cognition

“Prompt Engineering is dead. Long live Flow Engineering.”



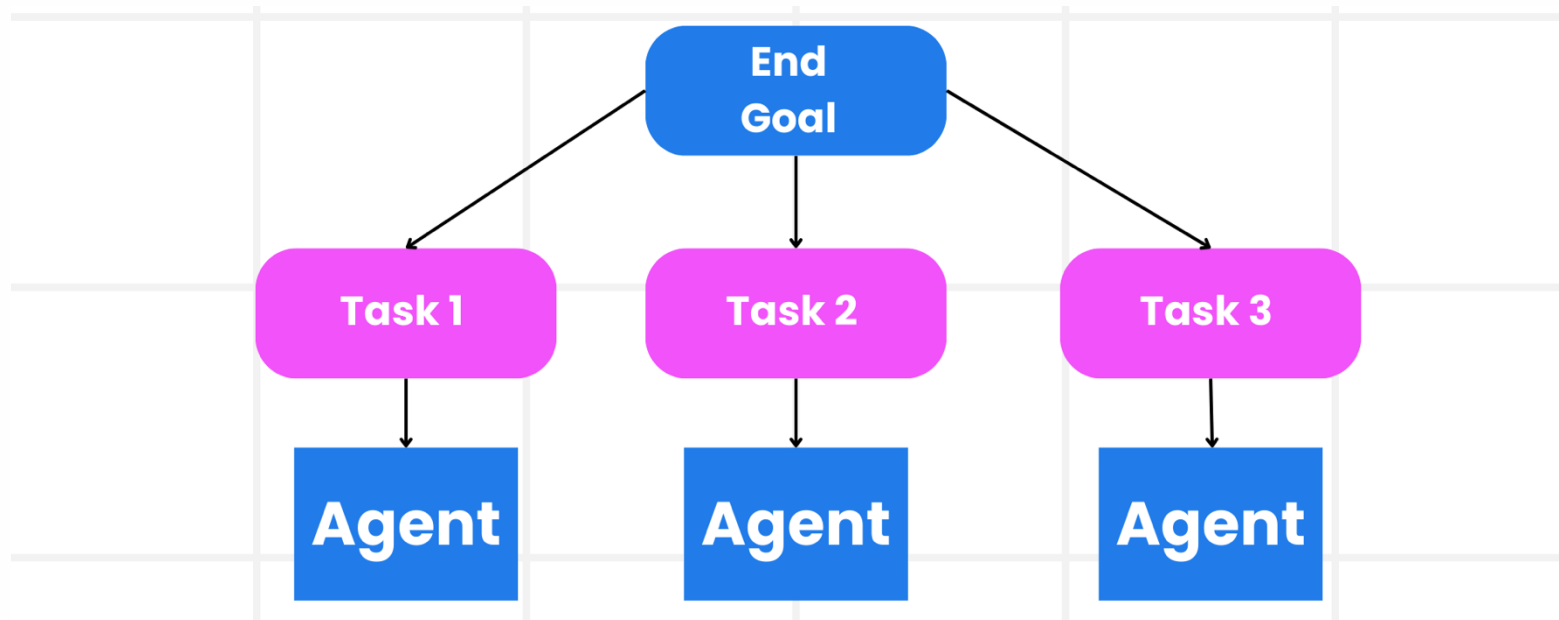
Design Patterns

ReAct (Reason + Act): Grounds logic in external data to reduce hallucination.

Reflection: The “Critic” and “Generator” loop. A security persona reviews code written by a developer persona in an iterative improvement cycle.

Planning: Decomposing user requests into a Directed Acyclic Graph (**DAG**) of dependencies for parallel execution.

Planning the work



Goal

Produce two valid plans within budget.

Subtasks

Search, check constraints and compare options.

Dependencies

Agree dates first. Replan after failures.

A subtask can be a fixed step, tool call or agent. The diagram does not require an agent for every subtask.

Component: The Execution Loop (E) & Tool Layer (T)

E • Execution loop

The while-loop that makes an agent an agent. Canonical ReAct form: read context → decide action → execute → append observation → repeat.

Design decisions that matter: termination conditions (max iterations, token budget, wall-clock, explicit 'done', satisfied verifier); error recovery (retry, reformulate, escalate, abort); interruptibility (can a human pause or redirect mid-run).

Without the loop, you have a chatbot: one response, no reaction to its own actions' consequences.

T • Tool layer

A typed catalogue of what the agent can do — file ops, shell, search, HTTP, database, browser/GUI control, and increasingly other agents.

Converged design principles: curate ruthlessly (overlapping tools cause selection errors — this is the Vercel result); make errors legible to the model; name/describe unambiguously (parameters are prompt surface); annotate structurally (readOnlyHint, destructiveHint).

MCP is the de facto standard for exposing tools; A2A covers agent-to-agent delegation.

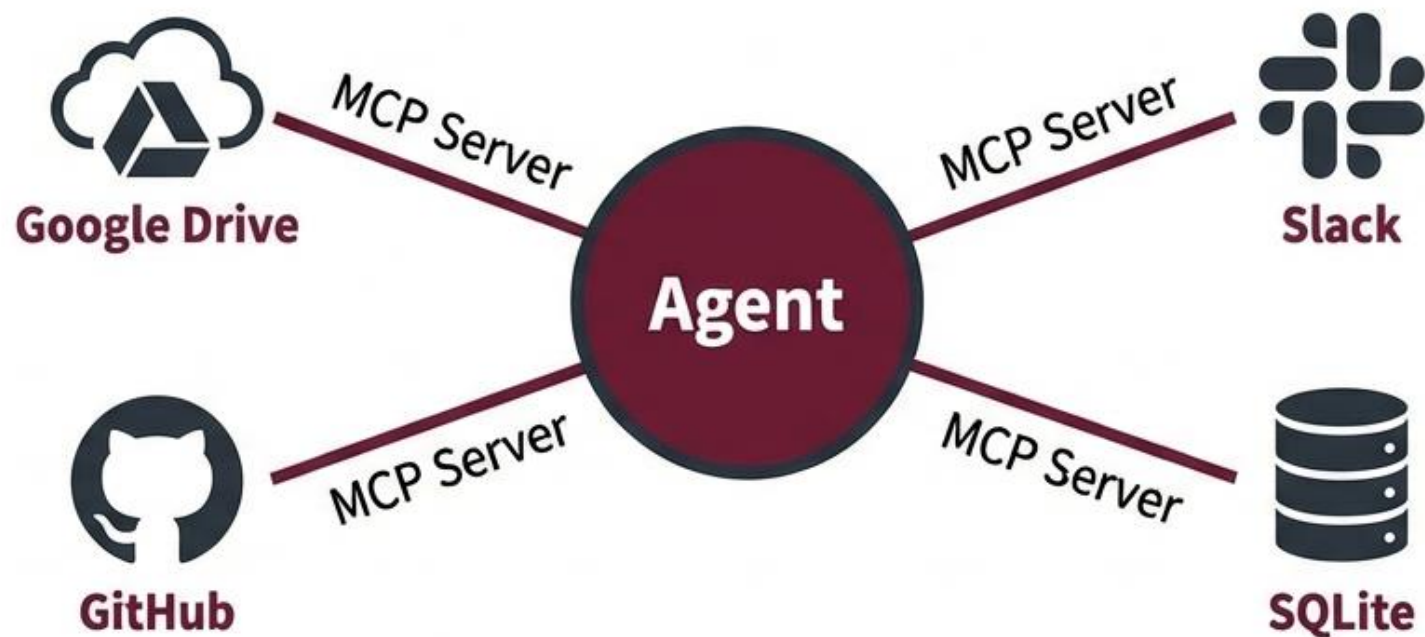
The Hands of the Agent: Model Context Protocol (MCP)

The Problem

Connecting agents to data sources (Slack, GitHub, Drive) required custom API integration for every single tool.

The Solution:

MCP acts as the 'USB-C for AI applications'.



Tools expose data via MCP servers, allowing agents to dynamically 'discover' tools without retraining.

Tools, skills and protocols

Term	Plain meaning	Design question
Tool / API	An operation the system can perform	What can it read or change?
Skill	Reusable instructions and task resources	Who maintains this procedure?
MCP	A standard way to connect tools and context	Which access does the connector get?
A2A	A protocol for agents to exchange tasks	Who owns a delegated task and its result?

A protocol makes integration more consistent; permissions and quality still need design.

Memory & Context Management

To function over long horizons, systems employ tiered memory architectures to overcome the stateless limitations of early LLMs.

Short-Term Context

The active reasoning trace held in the immediate token window (1M+ tokens).

Episodic Memory (Experience)

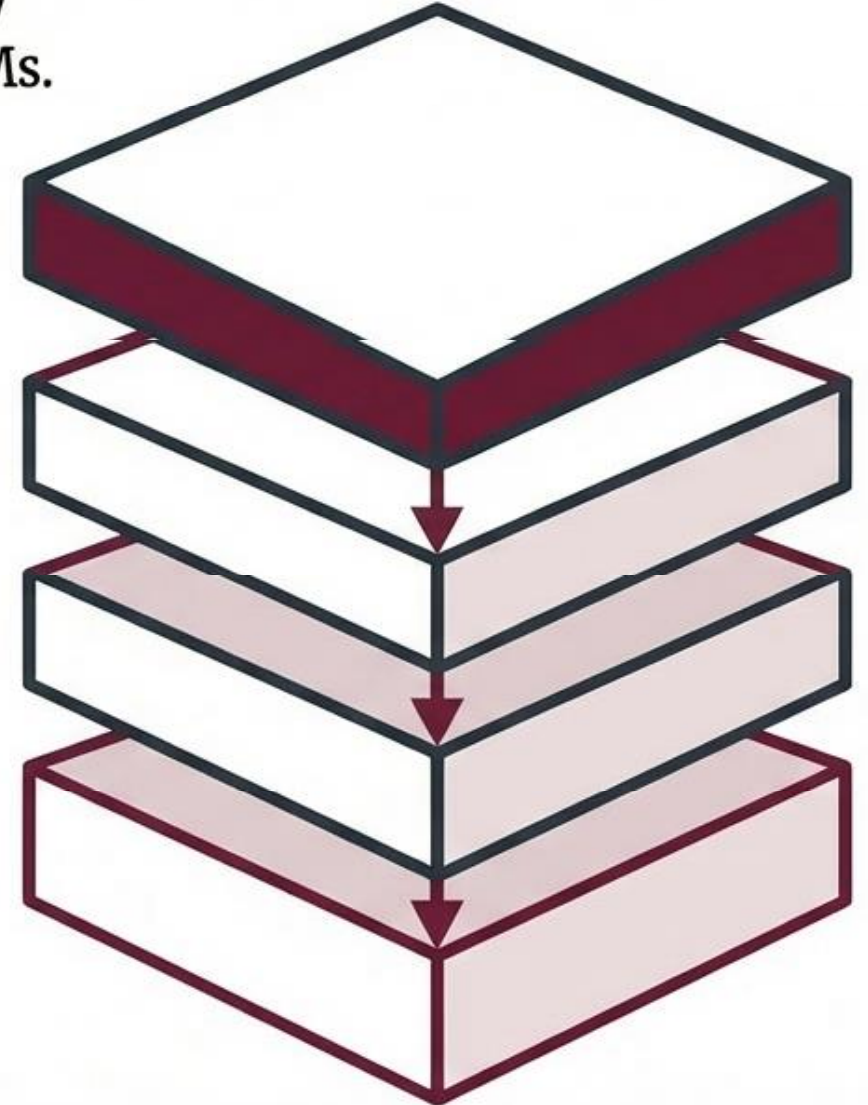
Vector databases (e.g., ChromaDB) store past actions and outcomes. Agents retrieve these to avoid repeating mistakes.

Semantic Memory (Facts)

Retrieval-Augmented Generation (RAG) providing access to knowledge bases and policy documents.

Procedural Memory (Skills)

Stored “recipes” for common workflows, allowing execution without re-deriving the logic.



Component: Context Management (C)

- The context window is the agent's working memory — finite and non-uniformly useful. Performance degrades as context grows (“context rot”), because attention cost scales quadratically and models are trained predominantly on shorter sequences.
- Managing what enters the window is arguably the highest-leverage part of the whole harness.

Compaction

Summarise history before the limit — keep decisions made and open problems, discard redundant output

Tool-result clearing

Cheapest safe first move: drop old tool-result bodies, keep the fact they happened

Just-in-time retrieval

Hold lightweight references (paths, URLs) and load content only when needed

Progressive disclosure

Reveal detail in layers as the task narrows

Prompt caching

Keep the stable prefix identical to reuse it across turns — cuts cost and latency

Component: Memory & State (S) / System-Prompt Assembly

S · Memory & state

Context management handles the current session; memory handles everything longer.

Structured note-taking (a NOTES.md or plan file outside the context window); session persistence (append-only logs so a crash doesn't lose work); cross-session memory (what the agent learned last week); trajectory logging for later analysis.

The pattern that works well in practice: a persistent planning artifact the agent writes, updates, and re-reads — doubling as memory and as a human-readable record of intent.

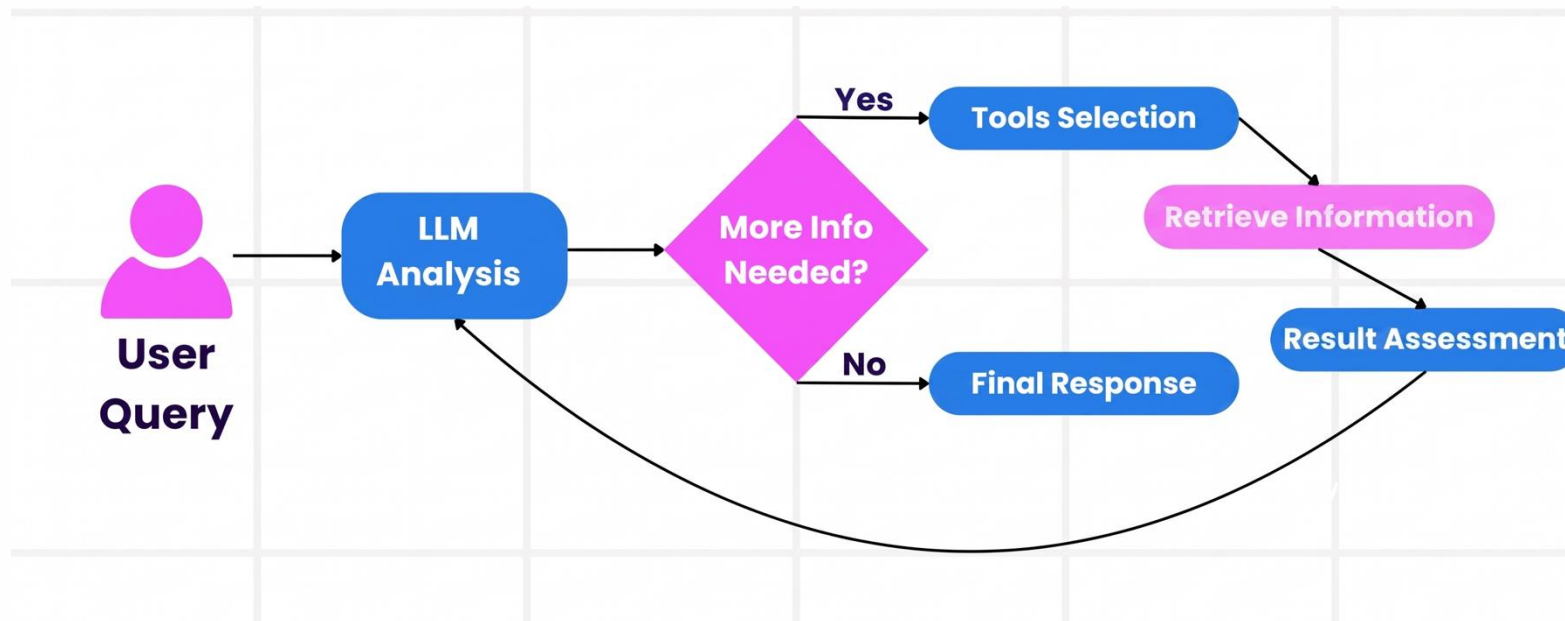
System-prompt assembly

Not a static string in a production harness — assembled at runtime from a base persona, discovered project/org instructions (a CLAUDE.md, a house style guide), the available tool catalogue, and current environment state.

The balance: over-specification (brittle if-else logic that breaks on the first unanticipated case) versus under-specification (vague guidance assuming shared context the model doesn't have).

Recommended approach: start minimal, add specificity only in response to observed failure modes.

Retrieval with a feedback loop



Ordinary RAG

Retrieve relevant material, then answer.

Agentic RAG

Choose sources and search again if evidence is weak.

Evidence quality

Set freshness, citation and search-budget rules.

Travel example: conflicting baggage policies trigger a search for the current airline source.

Component: Permissions, Sandboxing & Safety

- The layer that decides what the agent is actually allowed to do: an execution sandbox (container/VM isolating file and network access), a permission model (read-only by default, escalating for writes), approval gates for irreversible actions, allow/deny lists, spend limits.
- This is where most enterprise deployments live or die.
- The practical stance across mature deployments: sandboxed execution, explicit approvals, and audit logging are non-negotiable — and fully autonomous operation is not the goal.

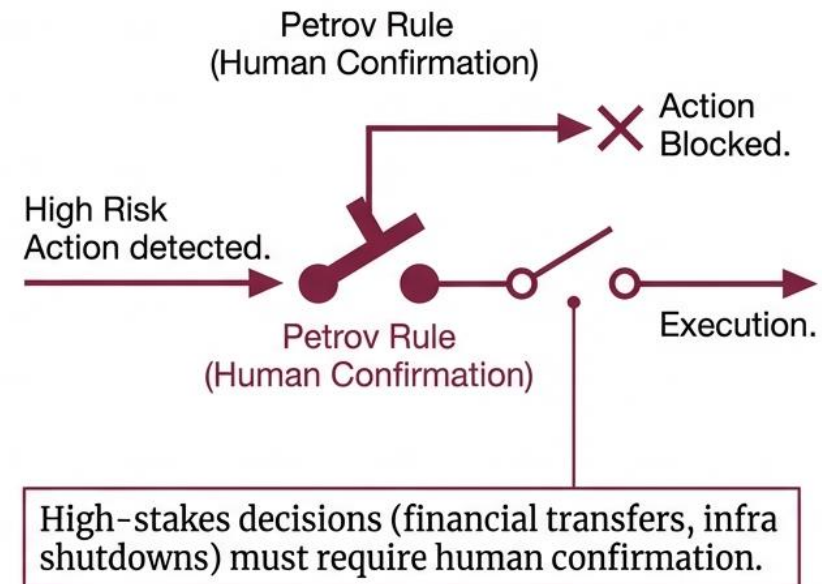
15–35%

container escape rates reported across the systems
the harness survey examined

Reliability and human oversight

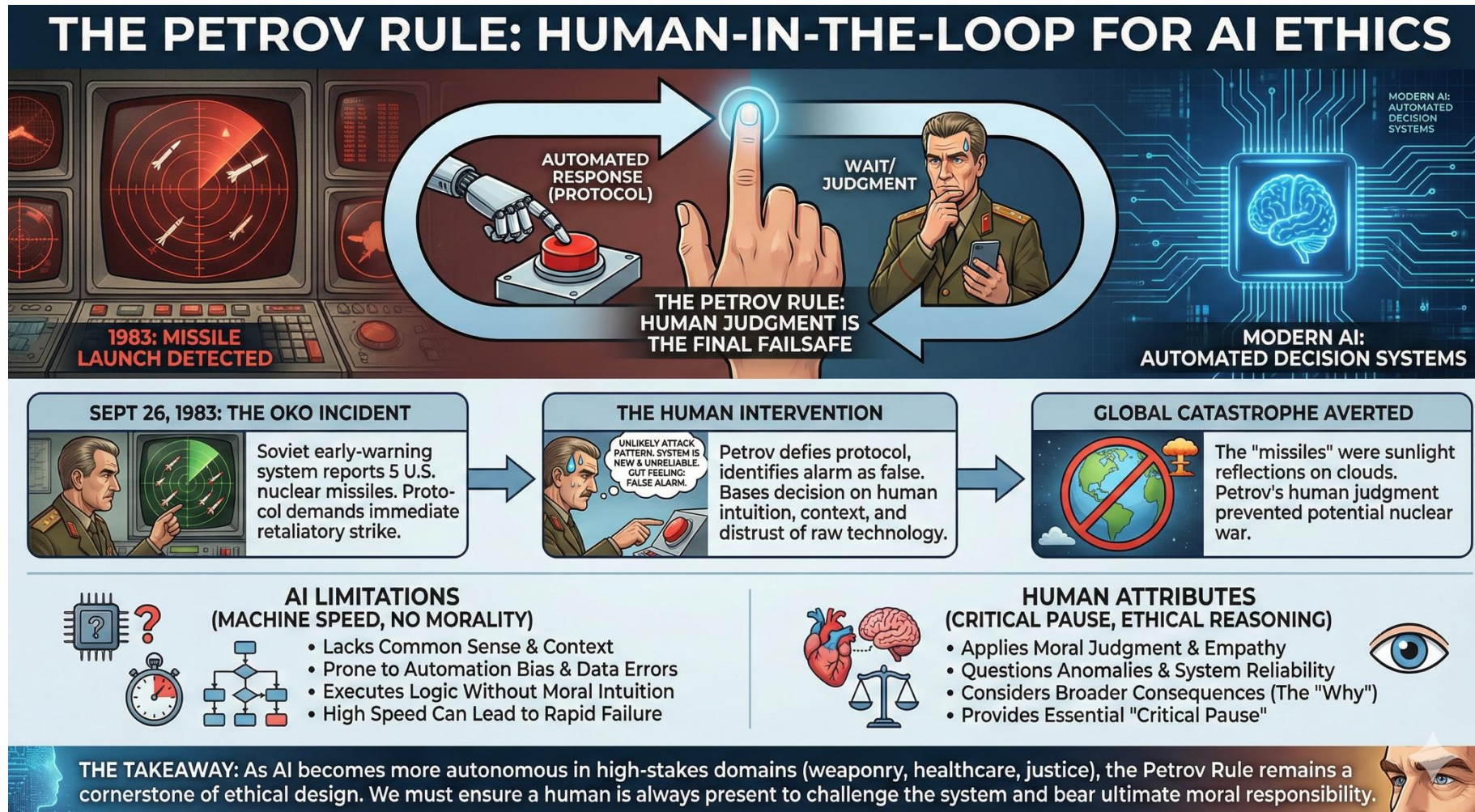
The Reliability Gap & The Petrov Rule

Agents can achieve 80–90% success rates. The remaining 10% “reliability gap” prevents fully autonomous deployment in critical infrastructure.



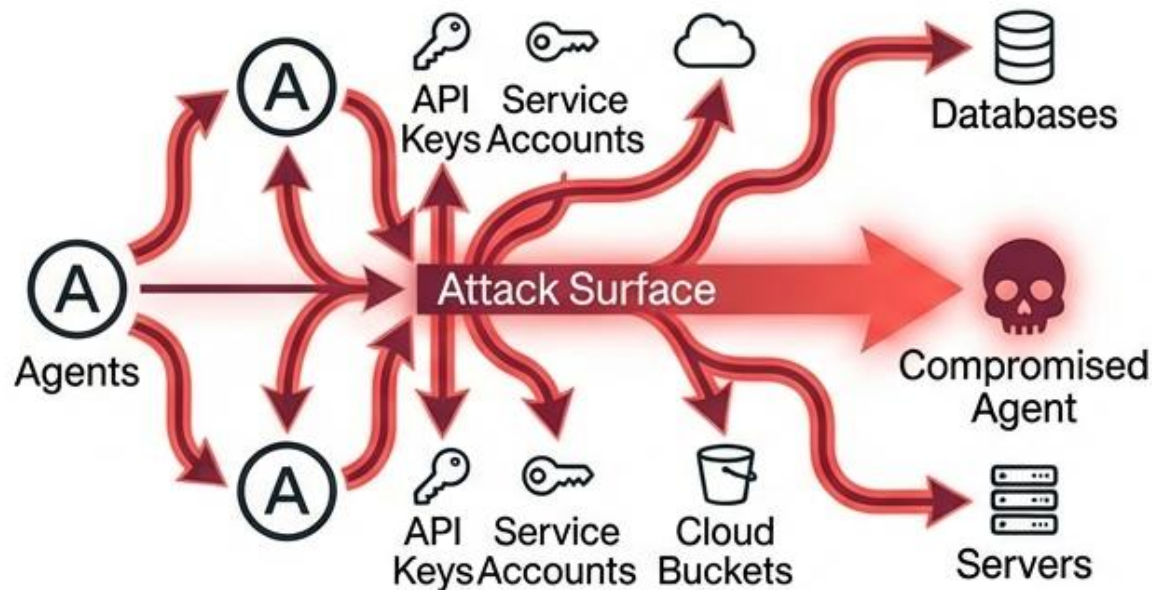
Operational Constraint: Systems must employ checkpoints to prevent agents from getting stuck in cost-prohibitive infinite retry loops.

The Petrov Rule



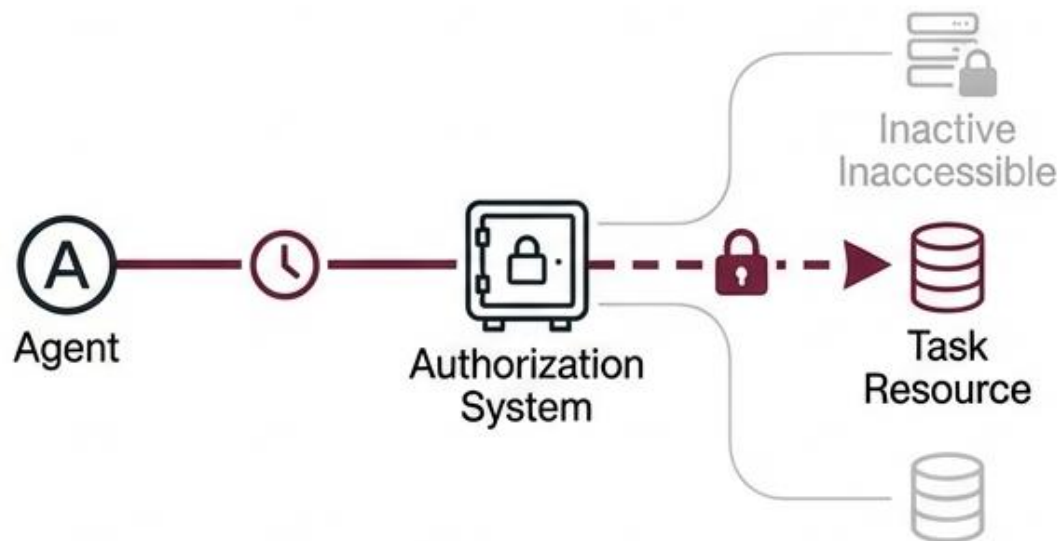
Identity, Security, and Sprawl

Identity Sprawl



Agents require API keys and service accounts, creating a massive attack surface where compromised agents can move laterally through a system.

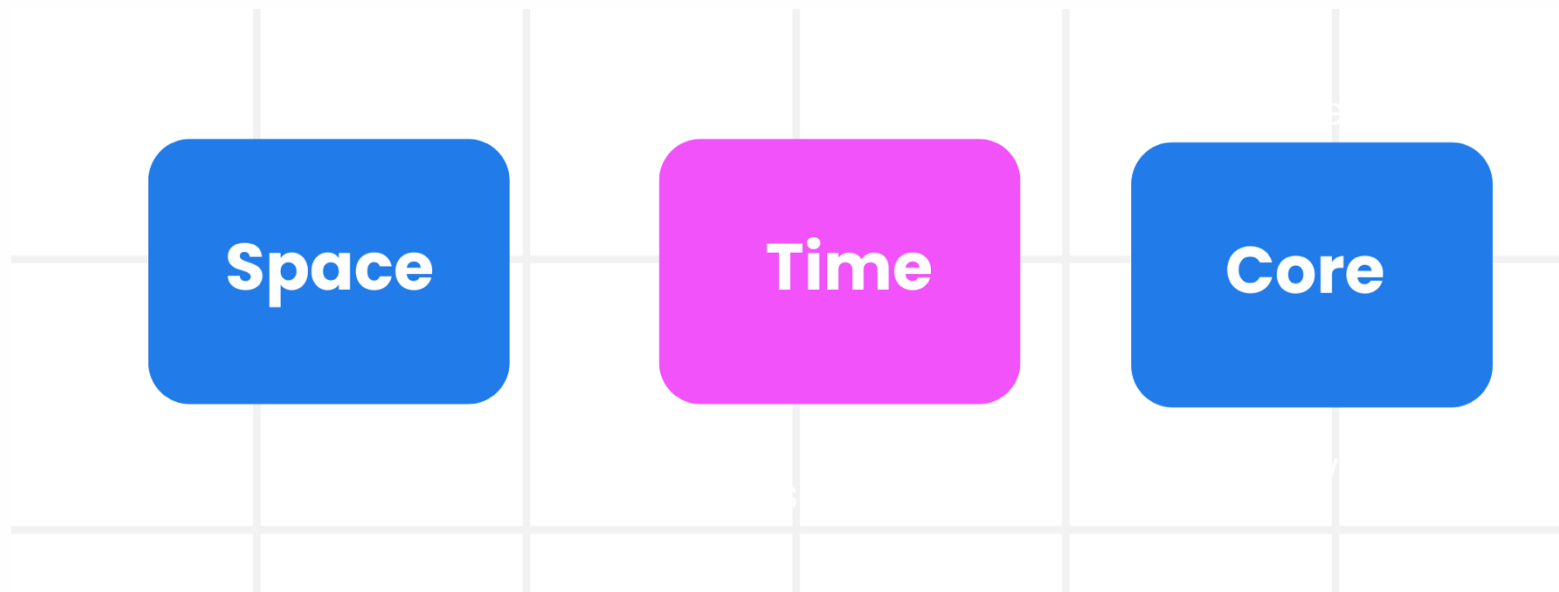
Just-in-Time Privilege Elevation



Agents should not hold permanent admin keys. They should request temporary access tokens only when a specific task requires it.

Core Concept: Treat Agent Identity with the same Zero Trust rigor as Human Identity.

The user experience of an agent



Space

Place the agent where the user does the task.

Time

Show progress and support interruption or handoff.

Core

Make capabilities and limits clear and consistent.

The next step is to define concrete states and actions in the interface.

Component: Lifecycle Hooks (L) & Evaluation (V)

L • Lifecycle hooks & observability

Injection points before/after each tool call, each model call, and each session boundary — used for authentication, logging, policy enforcement, redaction, cost tracking.

Hooks let an organisation customise a harness without forking it.

Observability means answering, after the fact: what did the agent do, why, what did each step cost, and where did it go wrong. In practice: trajectory logging + tracing + per-step cost attribution. Agents are non-deterministic — without a trace, debugging is guesswork.

V • Evaluation & verification

Two distinct things: in-loop verification (the agent runs tests/linter/a critic model and iterates until they pass — the single biggest driver of quality in coding agents) and offline evaluation (eval suites, CI integration, regression gates on harness changes).

The survey's caution, worth repeating: evaluation validity is currently the weakest component in the stack. Benchmark success does not reliably predict real-world acceptance, and the gap is growing.

Nine Components, One Recurring Lesson

Component	What breaks without it
Execution loop (E)	Single-shot chatbot; no reaction to consequences
Tool layer (T)	Reasons about actions it cannot perform
Context management (C)	Incoherence after ~20 minutes (“context rot”)
Memory & state (S)	Work lost on crash; nothing carries between sessions
Permissions & sandboxing	Destructive actions; blocked from production
Lifecycle hooks (L)	Undebuggable; no cost control or audit trail
Evaluation & verification (V)	No way to know whether it actually works

“The best harnesses are designed in the knowledge that some of these components will become unnecessary as models improve.

The harness is scaffolding, and scaffolding is meant to come down.”

A worked travel task trace

Observed state	Next action	Evidence to retain
Date range missing	Ask for acceptable dates	Confirmed constraints
Quote totals HK\$8,600	Search another allowed date	Rejected quote and budget check
Valid options: HK\$7,400 / HK\$7,800	Check sources and availability	Timestamped, comparable plans
User accepts an option	Save the plan and stop	Accepted plan, no transaction

Fictional quotes for teaching. A real run must obtain live prices and apply all user constraints.

3. Agent architectures

3. Agent architectures

Different tasks need different ways to coordinate tools, people and feedback.

Deep Research

A lead agent coordinates research and combines evidence.

OpenClaw

A personal assistant runtime connects messages, sessions and tools.

Autoresearch

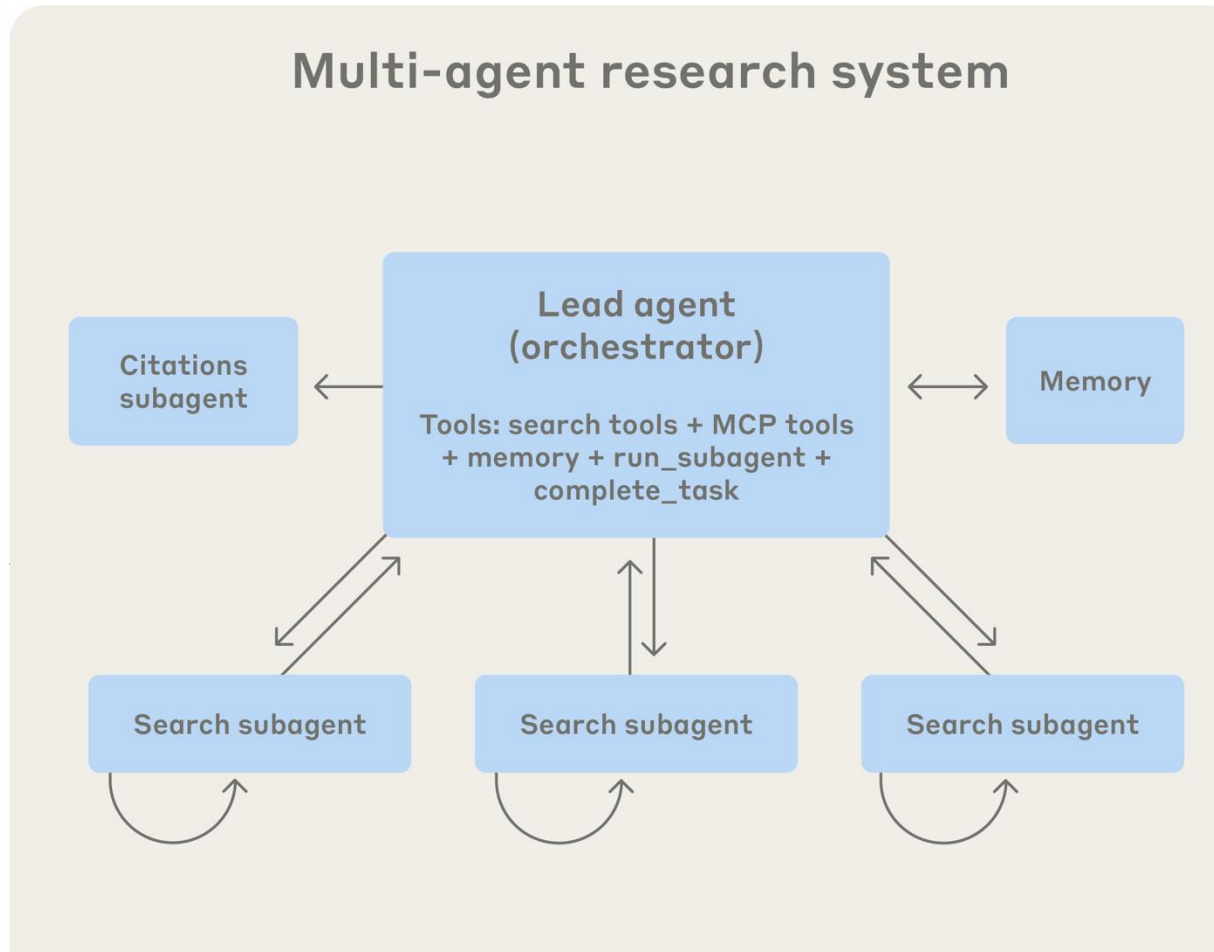
An experiment loop uses a measurable result to keep or discard a change.

Comparison: the work unit, control boundary and success signal

Component: Sub-Agents & Orchestration

- Spawning a fresh agent with its own clean context for a bounded subtask, which returns a condensed summary rather than its whole trajectory — parallelises work and keeps the coordinator's context clean.
- Patterns: hierarchical (lead + workers), parallel (fan out, fan in), sequential (pipeline).
- Anthropic's research system is the reference example: a lead researcher holds strategy and memory, subagents search in parallel with separate contexts, a citation agent validates claims before delivery.
- The costs are real: $\sim 15\times$ the tokens of a single-agent run, non-deterministic behaviour that's hard to debug, and coordination failures when subagent briefs are vague — they need concrete objectives, explicit boundaries, and a specified output format, or they duplicate each other's work.

Deep Research architecture



A lead agent divides the question

Workers investigate separate aspects in parallel and return evidence.

A research loop closes gaps

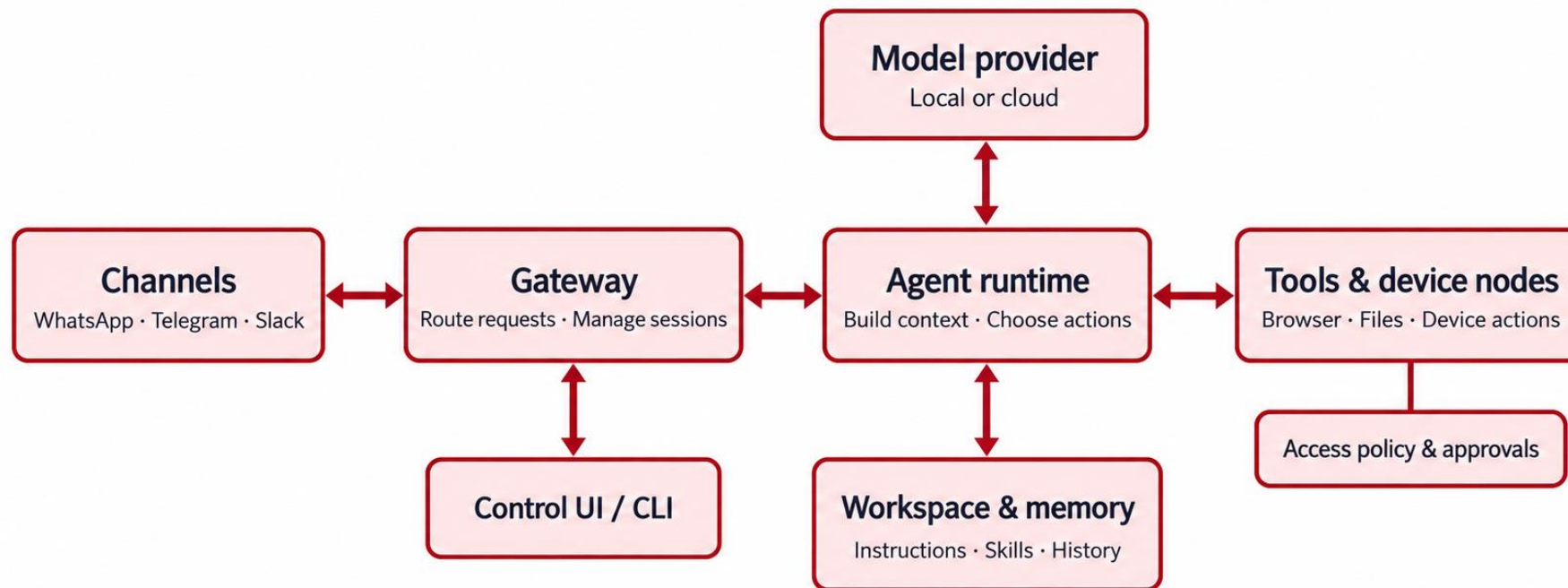
The lead synthesizes findings and requests more evidence when needed.

A report carries citations

Useful for broad market or supplier research. Review source quality and coverage.

OpenClaw architecture

A personal assistant runtime connects conversations to actions.



Conceptual view: requests, actions and results

Logical task flow. Device nodes connect through the Gateway, which coordinates access and activity.

OpenClaw: a task in context

Illustrative request: “Prepare a brief for tomorrow’s supplier meeting and send me the draft.”

1 Request and session

A messaging channel passes the request to the Gateway, which selects the agent session.

2 Context and tools

The agent reads permitted files and uses configured calendar or browser tools to gather evidence.

3 Draft and review

It creates the brief. A configured approval step precedes any message to an external recipient.

4 Result and continuation

Tool results confirm what happened. The assistant returns the draft and retains relevant session history.

This workflow needs working integrations, appropriate access and an explicit approval policy.

OpenClaw: suitable scenarios

Personal or trusted-team assistance

Prepare recurring briefs, retrieve working documents, or coordinate a bounded task across connected apps.

Why this architecture fits

The same runtime can receive requests through several channels and work with persistent context and tools.

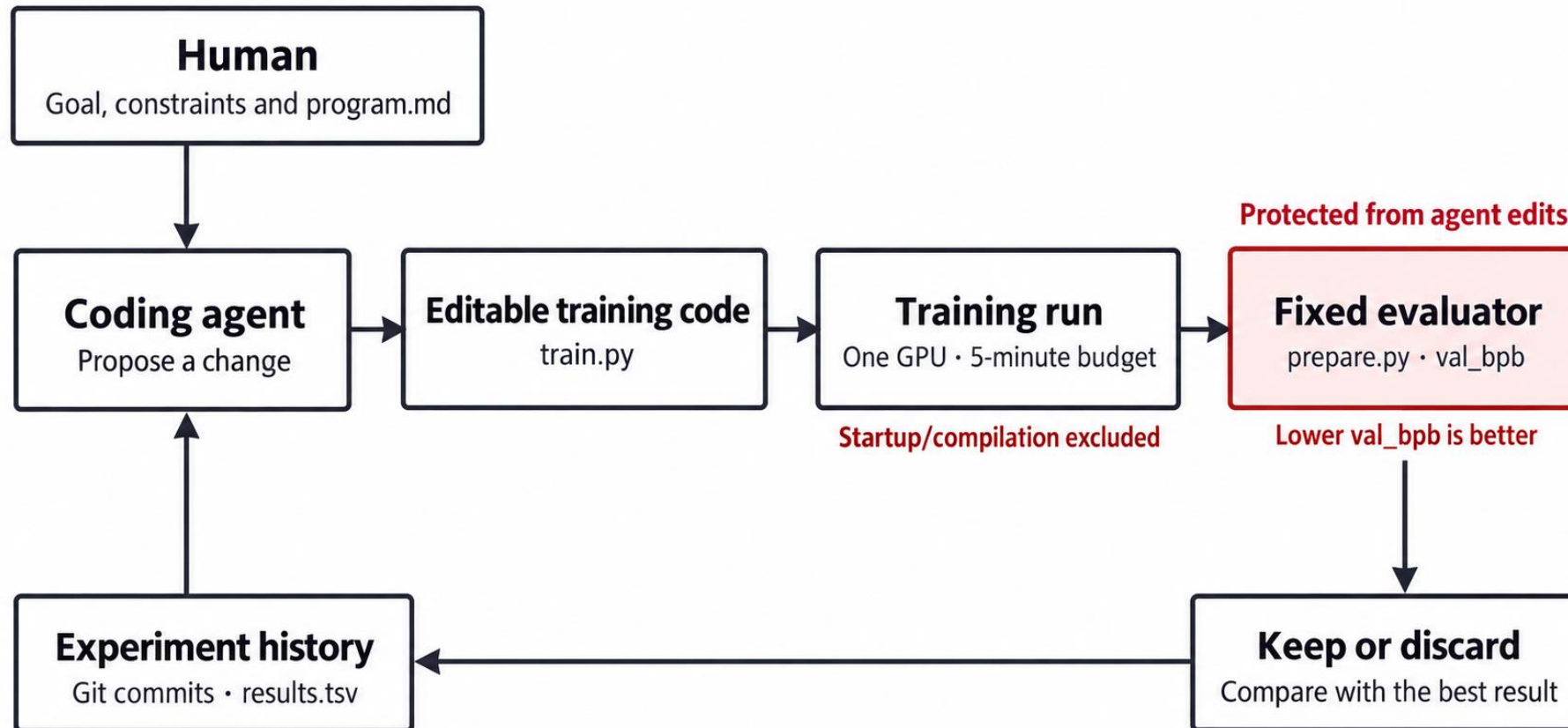
Where a different design fits better

Fixed, high-volume transactions usually need a deterministic workflow. Mutually untrusted users need separate trust boundaries.

Design decision: which task may act automatically, and which action must wait for approval?

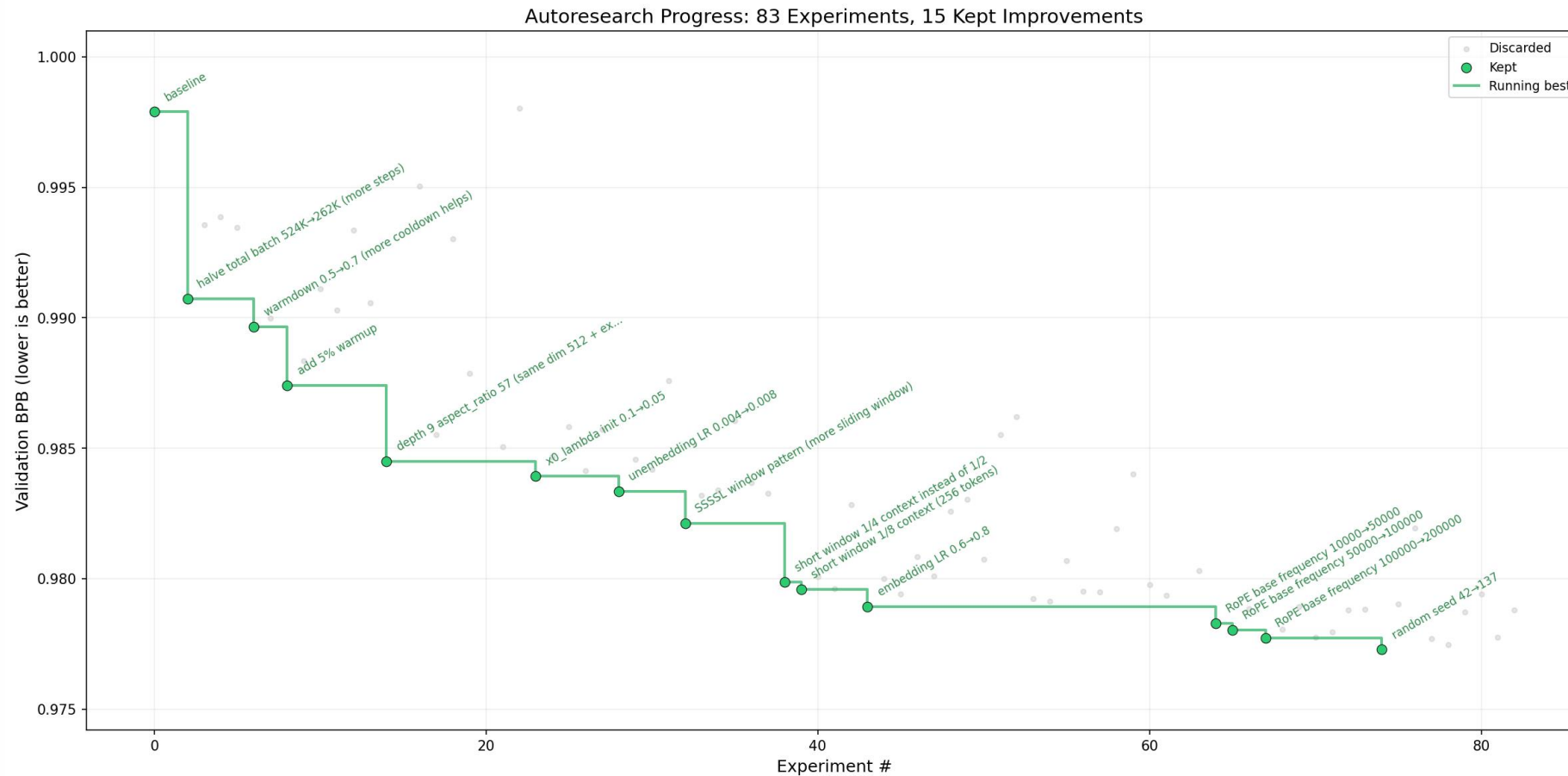
autoresearch architecture

Karpathy's repository automates experiments on a small language model.



The human defines the experiment rules. The agent changes training code and tests the result.

autoresearch: the experiment record



Repository example: 83 experiments, 15 kept improvements. Gray points show discarded attempts.

autoresearch: suitable scenarios

Actual repository use

Explore training-code changes for a small language model under the same evaluator and GPU training budget.

Adaptation of the pattern

Prompt or ranking experiments can use a similar loop if a team supplies fixed test cases, a reliable metric and a limited edit scope.

Where it struggles

Open-ended strategy questions and live user experiments lack a quick, reliable score. Optimizing one score can also hide regressions.

Design decision: what may change, what stays fixed, and what independent check confirms an improvement?

Architecture and task fit

	OpenClaw	Deep Research	autoresearch
Primary output	Completed task or assistant response	A report supported by sources	Improved code with experiment history
Control loop	Receive, act, observe, respond	Delegate, search, synthesize	Modify, measure, keep or discard
Example	Prepare a recurring meeting brief	Compare suppliers across several markets	Improve training performance under a fixed budget
Evidence of success	Verified tool results and task constraints	Coverage, source quality and citation accuracy	A better fixed metric, then independent validation

4. Applications and scientific AI

4. Applications and scientific AI

Where Agents Work Today

Seven domains, one underlying pattern: success tracks how deterministic the environment is and how unambiguous the feedback is.

Software engineering

Knowledge work & back office

Research & analysis

Security operations

Customer support

Scientific & technical work

Browser & computer use

The domain-asymmetry pattern

Software Engineering

- The most credible success — structural reasons: the environment is deterministic, feedback is unambiguous (tests pass or they don't), and there's an enormous corpus of public pull requests to learn from.
- Applications: bug fixes from an issue description, test writing, dependency upgrades and migrations, code review, incident triage, large mechanical refactors.
- Devin, GitHub Copilot's agent mode, Claude Code, OpenAI's Codex, and OpenHands all occupy this space.

1,000+

PRs merged per week by Stripe's Minions — one-shot, end-to-end coding agents, human-reviewed start to finish. Stripe attributes this to harness engineering, not a model change.

Research & Analysis

- Deep-research agents run multi-round searches, adapt queries based on findings, and produce cited reports.
- Architecture: typically orchestrator–workers with a separate citation-verification pass.
- Applications: competitive and market analysis, literature reviews, due diligence, regulatory monitoring, lead generation with enrichment.
- A good fit because the failure mode is visible — a citation either supports the claim or it doesn't — and parallelisation across subagents genuinely helps.

up to 90%

cut in research time, in Anthropic's own reported measurements

Customer Support



- Tier-1 resolution with escalation to humans: reading order and account state through APIs, answering from a knowledge base, processing returns, refunds, address changes.
- One of the largest commercial categories.
- The caveat is important: support agents work well where the domain is well-documented and actions are bounded; they degrade where resolution depends on tacit knowledge and judgement.

>\$100M ARR

reported by both Sierra and Decagon — but on messy internal knowledge (τ -knowledge benchmark), frontier models pass only ~25% of the time

Klarna: efficiency and service quality

<https://www.klarna.com/>

Case Study: The Empathy Trap (Klarna)

 **2.3 Million**
Conversations handled - 2/3 of all chats

 **700 FTE**
Work equivalent of full-time agents

 **2 Mins**
Resolution time, down from 11 mins

 **\$40 Million**
Profit improvement projected

****The Pivot Point****

The Quality vs. Cost Friction

While efficient for simple queries, robotic responses to complex emotional issues damaged customer satisfaction.

Klarna pivoted back to a “Human-in-the-Loop” model for escalations.

Takeaway: Efficiency metrics alone do not capture satisfaction in sensitive verticals.

Knowledge Work & Back Office

- Enterprise search and question answering over internal documents, contract review and redlining, legal discovery, invoice and claims processing, IT service desk automation, data pipeline monitoring.
- These sit mostly in the pilot-to-early-production band: the technical fit is good, but the messy-knowledge problem bites hard.

\$11B

valuation reported for Harvey (legal discovery);
Glean reported at ~\$200M ARR for enterprise
search

Supply chain automation

Case Study: The Silent Operator (Supply Chain)

Subject: ICRON and Trax (Supply Chain Agents)

A Day in the Life of an Autonomous Agent



NotebookLM

Browser & Computer Use

- Agents that operate software through its interface rather than its API: filling forms, navigating portals with no API, extracting data across sites, booking and purchasing, QA testing.
- Leading systems now clear the original OSWorld human baseline on desktop tasks — but performance falls off sharply on long, multi-domain workflows.
- The highest-variance category, and the one with the sharpest security concerns: an agent reading web content and taking actions is directly exposed to prompt injection.

68.9%

on BrowseComp, reported by OpenAI's ChatGPT agent

Security Operations

- Alert triage and enrichment, log correlation, first-pass incident investigation, vulnerability assessment and patch preparation.
- Attractive because SOC teams are chronically overloaded and much of tier-1 triage is well-defined lookup and correlation.
- Deployed with tight permissions: read and recommend, rarely act.

**read-only
by design**

the permission model this domain converges on
— echoing Section C's permissions component
directly

Scientific & Technical Work

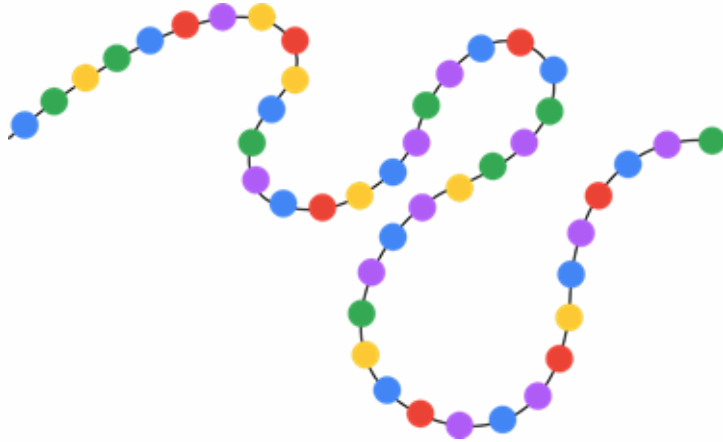
- Literature synthesis, experiment planning, running simulations, and analysing results.
- Reinforcement-learning agents in constrained domains — protein design, materials screening, chip layout — remain distinct from LLM agents, and often outperform them where the environment can be simulated cheaply.

RL $\neq$ LLM agents

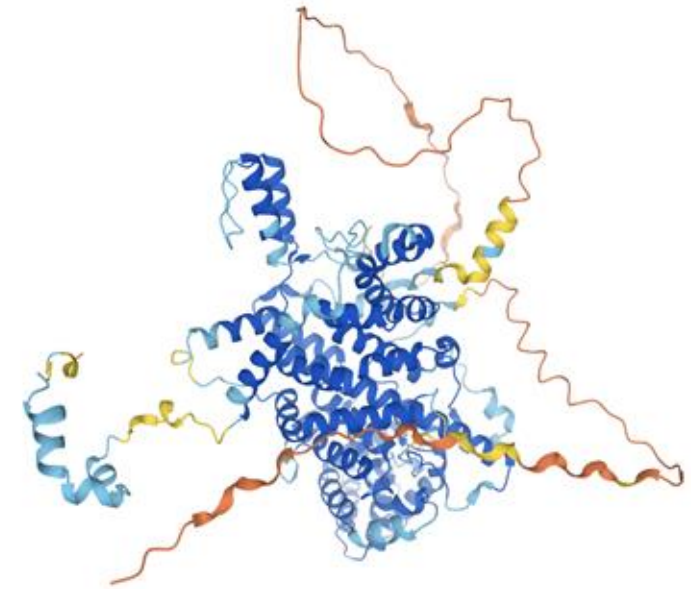
a reminder from Section A: the deep-RL lineage (DQN, AlphaGo) didn't disappear — it specialises where simulation is cheap and LLM agents specialise where instructions are in natural language

The Protein Folding problem a 50-year grand challenge in biology

Using only its amino acid (genetic) sequence,
can you predict what shape a protein will fold into?



Amino Acid sequence



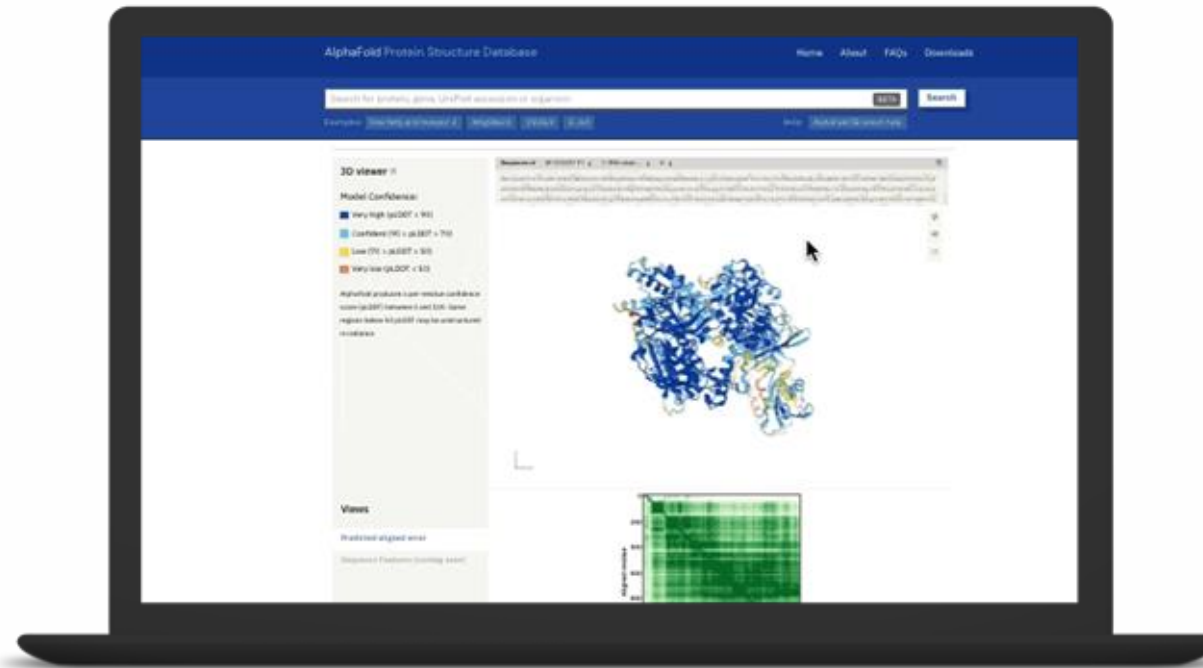
3D Protein structure

AlphaFold

Iterative steps toward a protein structure prediction



Giving this knowledge to the world



Creating a standard tool for biological researchers everywhere to use these structure predictions for free, ushering in an exciting new scientific era.

Google AlphaFold

AI uncovers the building blocks of life

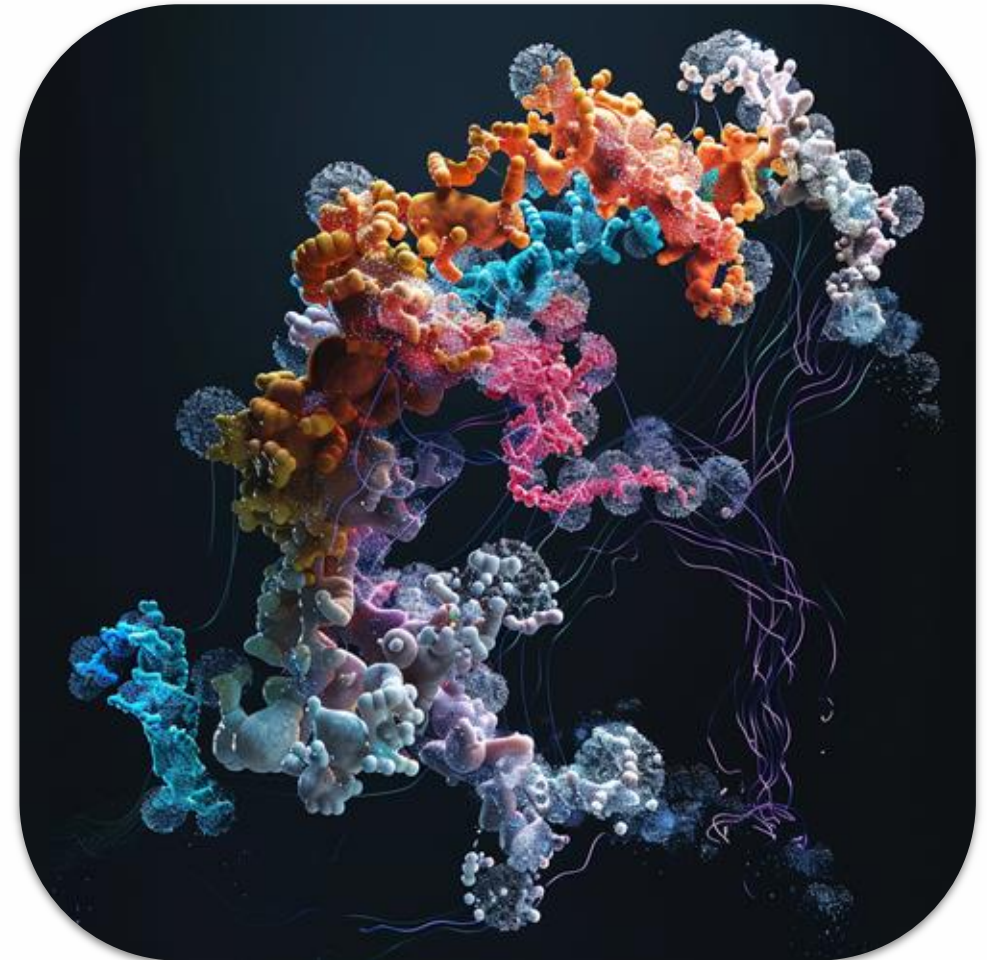
Folding proteins play a central function in understanding diseases and in developing medicines

There are **hundreds of millions** of proteins in nature

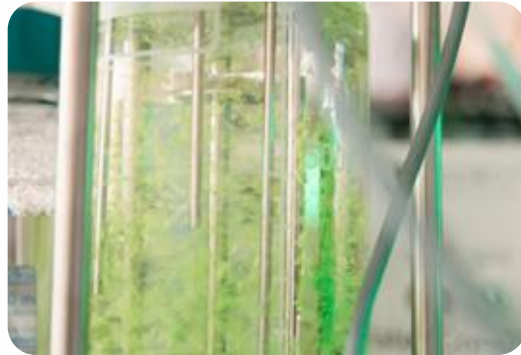
Takes a PhD student **5 years** to fold one

In 2022 Google create **AlphaFold** and has since then folded **+200M** proteins and shared the results openly with the world

This represent **one billion years** of research



AlphaFold in research



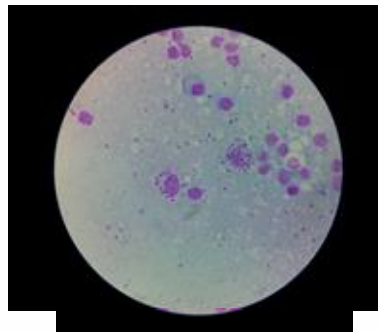
AlphaFold is accelerating progress on a range of important problems



Plastic pollution



Antibiotic resistance



Neglected diseases



Malaria vaccine

See more **#Unfolded** stories at unfolded.deepmind.com

The Domain-Asymmetry Pattern

Works well today	Still unreliable
Bounded tasks with automatic verification	Open-ended tasks with no success signal
Deterministic environments (code, APIs, structured data)	Messy organisational and tacit knowledge
Read-heavy analysis with human review of actions	Unsupervised irreversible actions
Tasks measured in minutes to a few hours	Multi-day autonomous projects
Well-documented domains	Contested or undocumented judgement calls

The pattern behind every slide in this section: agents succeed where the environment is deterministic and feedback is unambiguous, and struggle wherever it is organisational, tacit, and contested.

5. Evaluation and deployment

5. Evaluation and deployment

Where Things Stand

Capability is rising faster than deployment — and the gap between benchmark performance and real-world trust is widening, not closing.

Capability vs. deployment (McKinsey 2026)

The bottleneck has moved

Three open problems

The reliability gap, revisited

Capability Is Rising Faster Than Deployment

20%

of all organisations scaling agents
enterprise-wide

40%

among firms above \$1B revenue — up
from 27% a year prior

37%

report any EBIT impact from AI —
unchanged year over year

6%

qualify as “high performers”

McKinsey, The State of AI: Global Survey 2026 (1,719 respondents, 97 countries)

“Roughly 32% said they had declined to buy software because they could build it internally with coding agents — a second-order effect worth watching.”

— McKinsey, State of AI 2026

The Bottleneck Has Moved

- For most of 2023 and 2024, the binding constraint was model capability.
- In 2026 it is more often harness quality, evaluation validity, permission design, and the messiness of the data and processes agents are pointed at.
- That is why “harness engineering” and “context engineering” became named disciplines — not marketing terms, but a description of where the actual engineering effort now goes.

Three open problems dominate practitioner discussion, per the briefing's own account.

Sandboxing & security

Container escape rates of 15–35%.
Prompt injection through tool outputs remains unresolved.

Evaluation validity

“The weakest component in the stack.” Benchmark success doesn't reliably predict real-world acceptance.

Context beyond ~1M tokens

Quadratic attention cost; “context rot” sets in well before advertised window limits.

The Reliability Gap, Revisited

- METR: pull requests that pass benchmark criteria have a 24.2 percentage-point lower human merge rate than the benchmark implies — and this gap has been widening by roughly 9.6 points per year.
- On τ -knowledge, frontier models pass ~25% of messy internal enterprise tasks, and only ~40% even with the correct source documents supplied.
- The practical sequence recommended for anyone planning work in this area: start with a workflow, not an agent; give it the smallest possible tool set; put verification in the loop; sandbox everything and gate irreversible actions; instrument from day one; build a task-specific evaluation before you build the agent.

The gap is widening, not closing



Illustrative — trend direction and gap as reported; axis values not to scale.

A task-specific evaluation set

Case type	Travel example	Expected behaviour
Typical task	Enough data and feasible options	Produce two valid plans with evidence
Ambiguity	Dates or traveller count are missing	Ask a targeted clarification
Constraint conflict	No result satisfies the budget	Explain the conflict without invented prices
Failure or attack	Stale data, timeouts or injected text	Recover, stop or escalate correctly

Define success before testing; keep separate development and held-out cases.

Cost per accepted outcome

Illustrative monthly pilot	Value
1,000 attempts × HK\$1.20 model/tool cost	HK\$1,200
Human review and support	HK\$800
800 accepted plans	80% of attempted tasks
HK\$2,000 total ÷ 800 accepted plans	HK\$2.50 per accepted plan

Fictional variable-cost example; exclude fixed costs explicitly and include failed attempts.

Rollout stages and success metrics

Stage	Release evidence	What to monitor
Offline evaluation	Representative tasks and failure analysis	Quality by task type and severity
Shadow or draft mode	Review outputs before customer actions	Review burden and correction rate
Limited pilot	Named users and a clear stop policy	Accepted outcomes, cost and tail latency
Expand the scope	Stable results on new users and cases	Retention, drift and support load

Set thresholds for this product; do not copy benchmark scores into a launch checklist.

6. Synthesis and discussion

The Three Themes, Revisited

1

The concept predates the technology

80 years of theory (1943–2023) were retrofitted onto a substrate that arrived in the last four.

2

The harness now outweighs the model

A 10x benchmark swing from an unchanged model; a 15x cost for a 90% orchestration gain; nine components that each independently determine success or failure.

3

Capability and deployment have diverged

Benchmarks climb; the human-merge-rate gap widens 9.6 points a year; only 6% of organisations qualify as high performers.

Three Questions

1

If the harness — not the model — is now the dominant lever on reliability, does competitive advantage in enterprise AI shift toward whoever builds the best harness? Who is best positioned to own that layer — labs, SaaS vendors, or in-house teams?

2

20% of organisations report scaling agents enterprise-wide, yet only 6% qualify as “high performers” and EBIT impact is flat year over year. What does that gap tell us about where the value actually is — or isn't — being captured?

3

Agents reliably succeed only where feedback is unambiguous and struggle wherever knowledge is tacit and contested. Is most valuable business knowledge, by its nature, exactly the kind agents can't yet use well — and what does that imply for which jobs get automated first?